BÀI THỰC HÀNH 3: QUẢN LÝ NHÂN SỰ BẰNG SQLITE

Họ tên: Trần Xuân Thành
MSSV: B23DCAT280 — Lớp: E23CNPM01
Ngày: 31/10/2026

1. Yêu cầu

Nhân sự gồm: Mã nhân sự, Họ và tên, Năm sinh, Giới tính, Quê quán, Trình độ, Ảnh, Mức lương, Phòng ban (Hành chính, Phần mềm, Kỹ thuật, Dự án). Viết ứng dụng quản lý nhân sự hiển thị danh sách nhân sự với các chức năng thêm, sửa, xoá, tìm kiếm, sắp xếp (theo mức lương) bằng SQLite. Các giao diện đều phải có thông tin cá nhân (MSSV - Họ tên). Ứng dụng lưu đủ ngày sinh (DoB) thay cho năm sinh.

Ứng dụng gồm 3 Activity theo sơ đồ trên lớp:

- Activity_main (MainActivity): MSSV và Họ tên trên 2 dòng, ô tìm kiếm + nút Search, nút sắp xếp theo lương, ListView danh sách nhân sự và nút ADD. Bấm 1 dòng để sửa, nhấn giữ 1 dòng để xoá.
- Activity_add (AddActivity): form nhập 9 trường, nút Submit ghi vào CSDL rồi quay về danh sách, nút Reset xoá trắng form.
- Activity_detail (DetailActivity): form đã điền sẵn thông tin nhân sự, nút Update ghi thay đổi vào CSDL. Mã nhân sự là khoá chính nên không cho sửa.

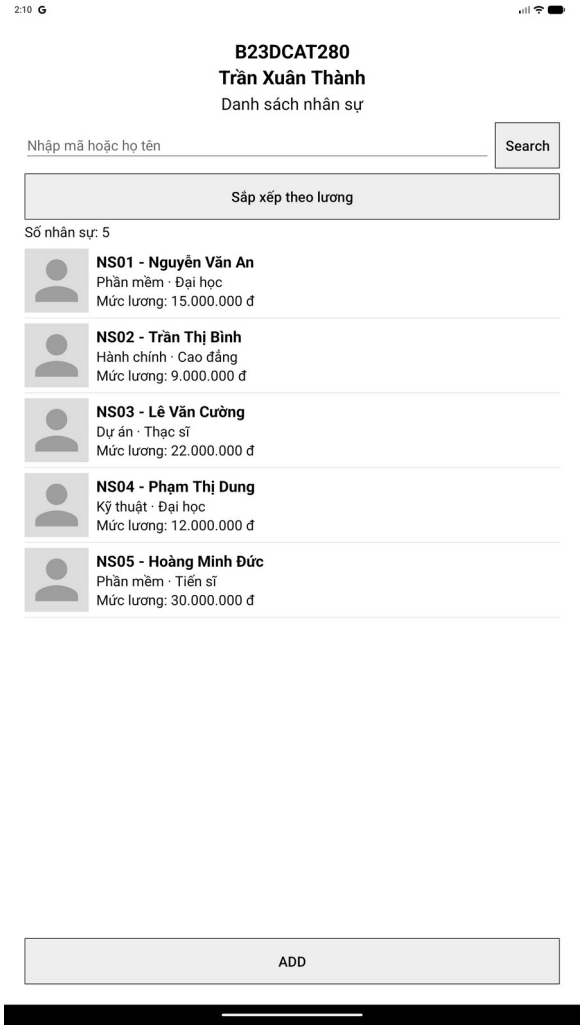
2. Cơ sở dữ liệu

Lớp DBHelper kế thừa SQLiteOpenHelper: tạo CSDL quanlynhansu.db, tạo bảng nhansu bằng execSQL và thêm sẵn 5 nhân sự mẫu. Các thao tác dùng query (danh sách, tìm kiếm với LIKE, sắp xếp với ORDER BY luong), rawQuery (lấy 1 nhân sự theo mã), insert / update / delete với ContentValues.

Cột	**Kiểu**	**Ý nghĩa / cách nhập**
ma	TEXT PRIMARY KEY	Mã nhân sự (EditText)
hoten	TEXT	Họ và tên (EditText)
ngaysinh	TEXT (yyyy-MM-dd)	Ngày sinh, chọn bằng DatePickerDialog, hiển thị dd/MM/yyyy
gioitinh	TEXT	Nam / Nữ (RadioButton)
quequan	TEXT	Quê quán (EditText)
trinhdo	TEXT	Trung cấp, Cao đẳng, Đại học, Thạc sĩ, Tiến sĩ (Spinner)
anh	BLOB	Ảnh chọn từ máy, thu nhỏ và nén JPEG (nút Chọn ảnh)
luong	REAL	Mức lương (EditText số)
phongban	TEXT	Hành chính, Phần mềm, Kỹ thuật, Dự án (Spinner)

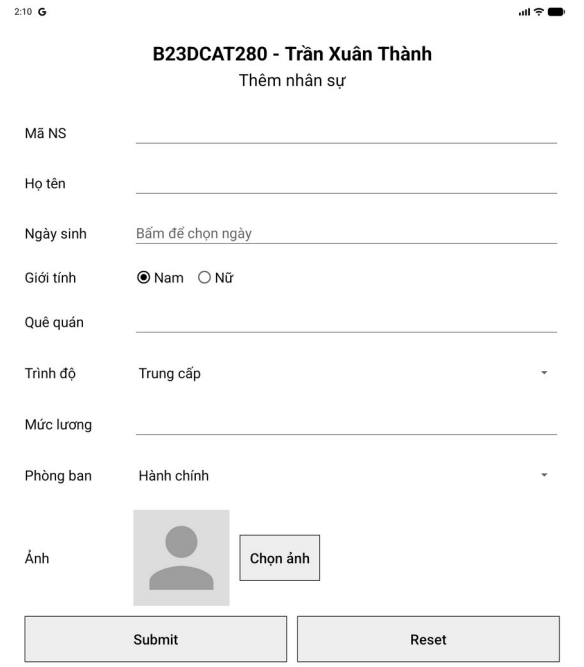
3. Giao diện khi chạy

3.1. Hiển thị danh sách

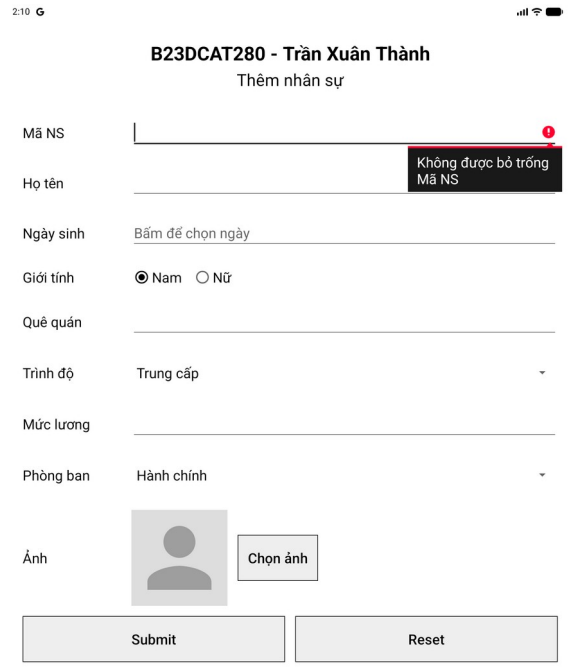


Hình 1. Activity_main: danh sách 5 nhân sự mẫu đọc từ SQLite

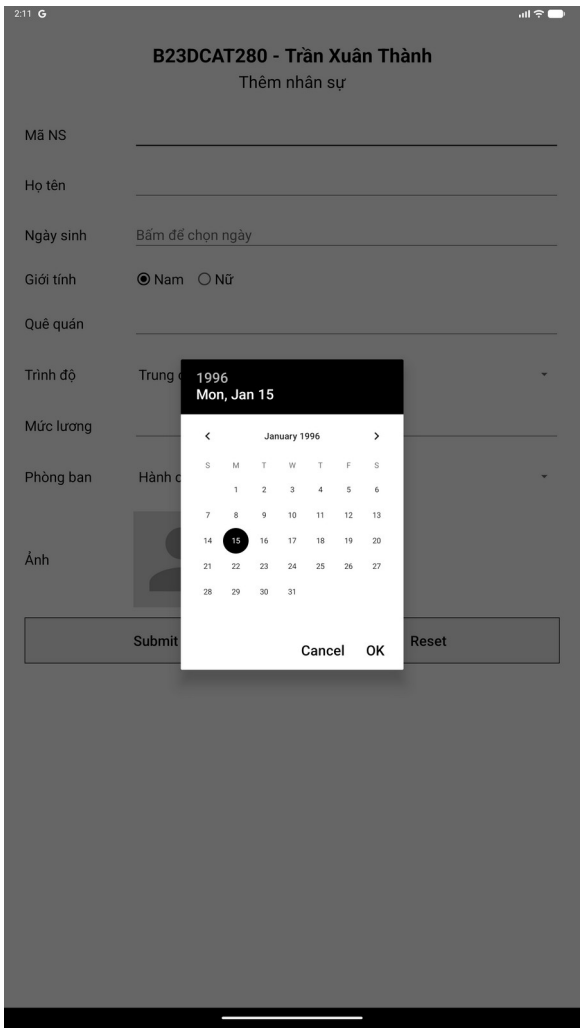
3.2. Thêm nhân sự



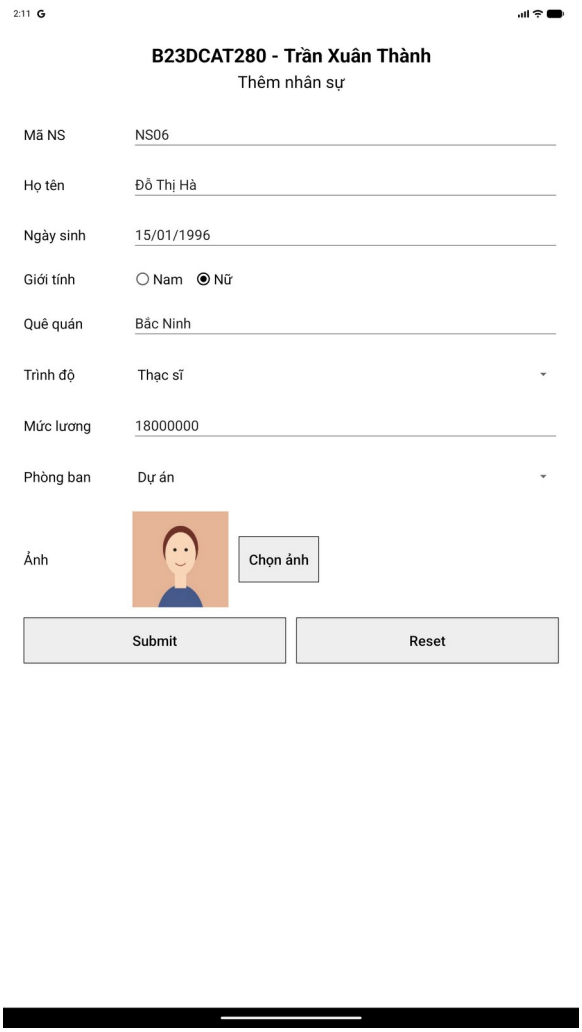
Hình 2. Bấm ADD: Activity_add với form trống



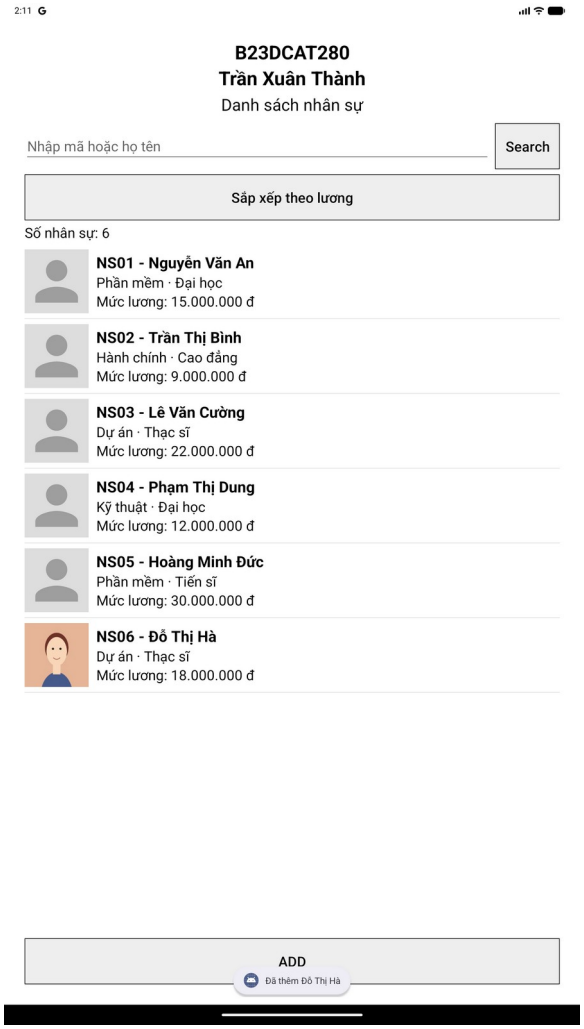
Hình 3. Bấm Submit khi bỏ trống: báo lỗi ngay tại ô Mã NS



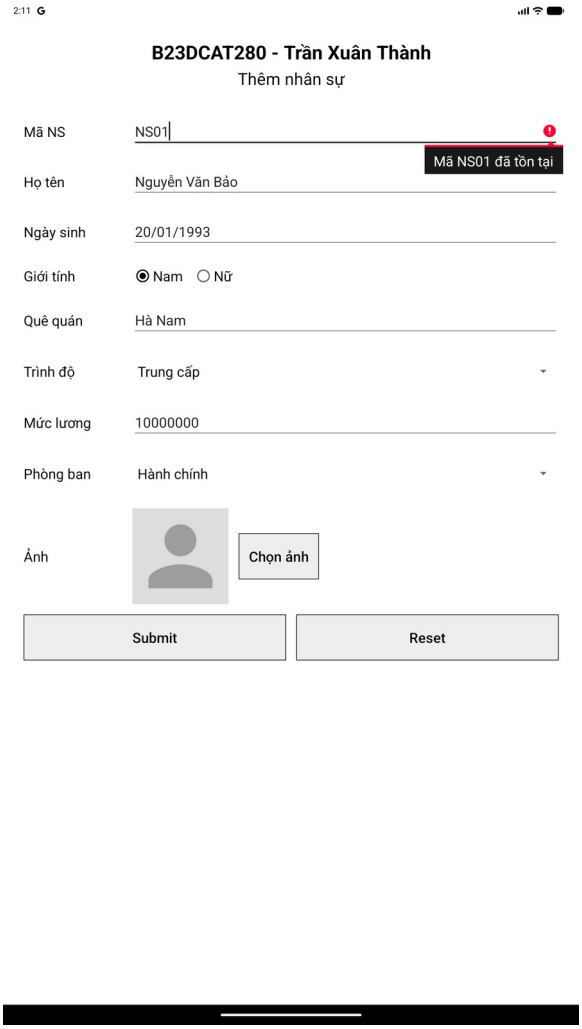
Hình 4. Bấm ô Ngày sinh: chọn ngày 15/01/1996 trên lịch



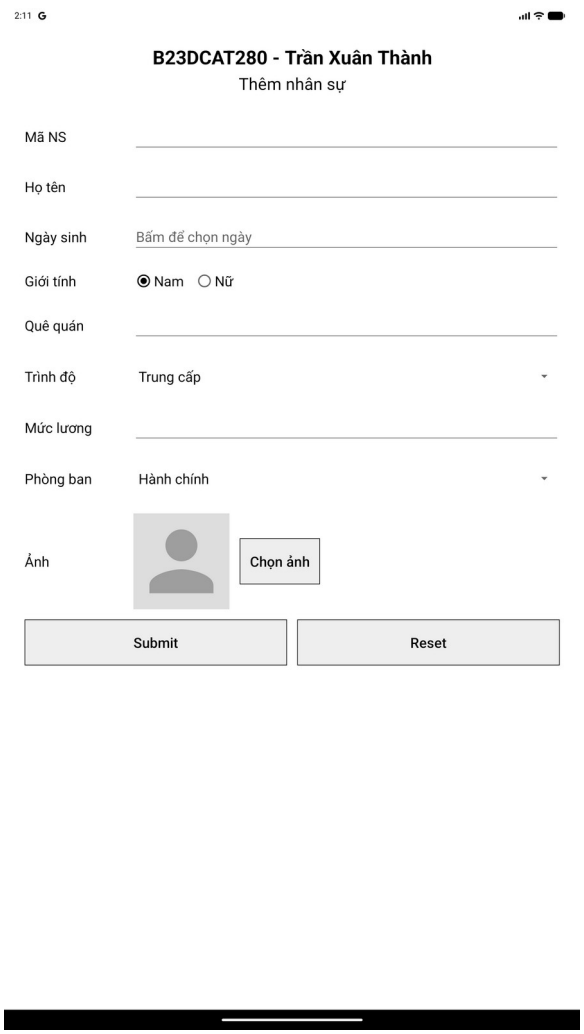
Hình 5. Nhập đủ thông tin nhân sự NS06 và chọn ảnh từ máy



Hình 6. Submit xong: quay về danh sách, NS06 có ảnh xuất hiện, Toast "Đã thêm Đỗ Thị Hà"

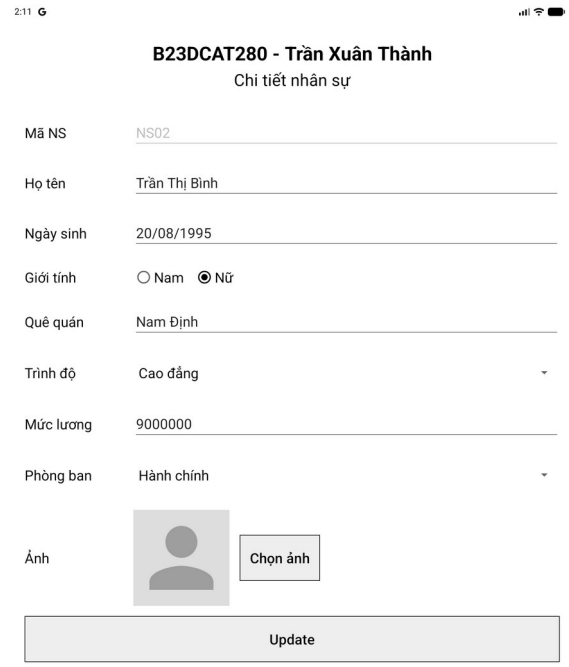


Hình 7. Thêm nhân sự trùng mã NS01: báo "Mã NS01 đã tồn tại", không ghi vào CSDL

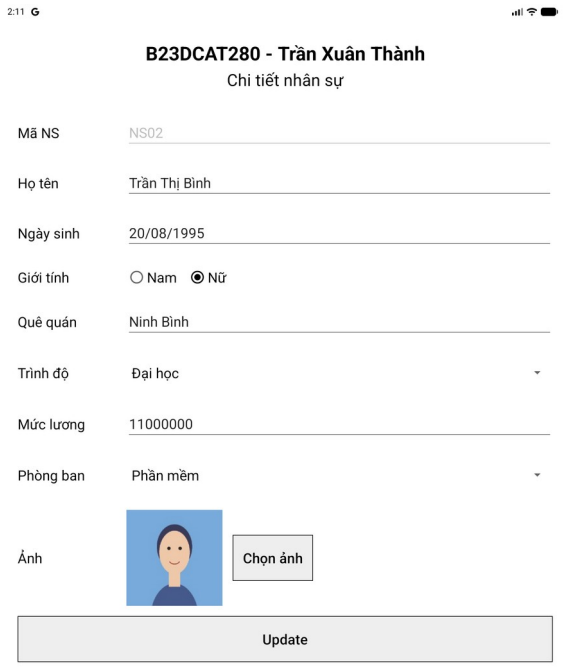


Hình 8. Bấm Reset: xoá trắng form

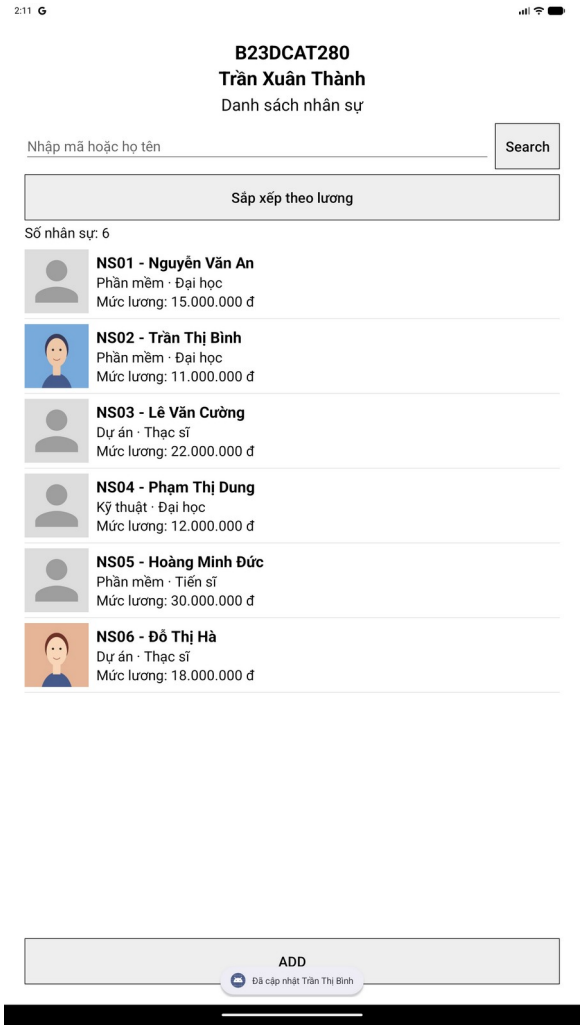
3.3. Sửa nhân sự



Hình 9. Bấm dòng NS02: Activity_detail điền sẵn thông tin, ô Mã NS bị khoá

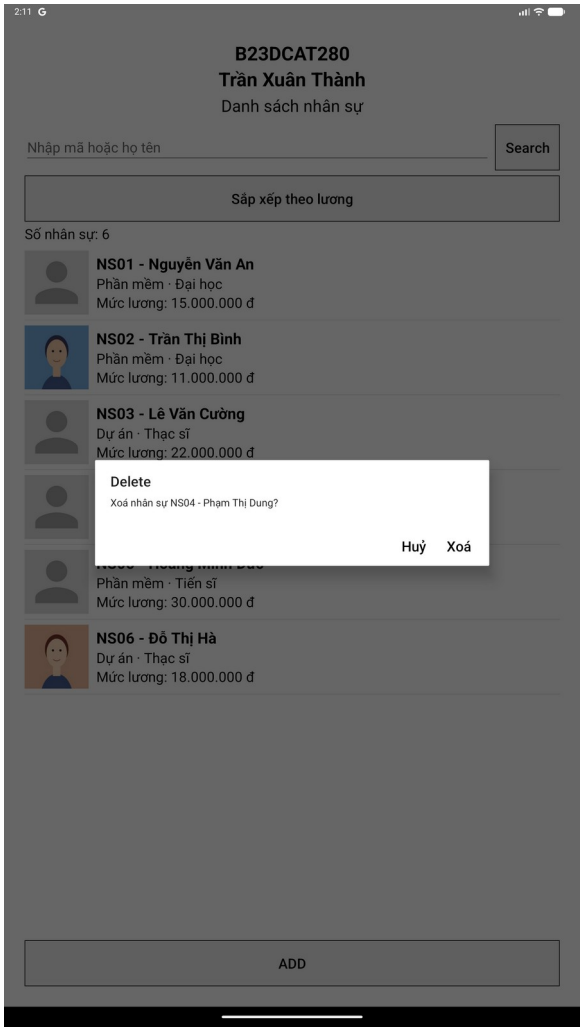


Hình 10. Sửa quê quán, trình độ, mức lương, phòng ban và thêm ảnh

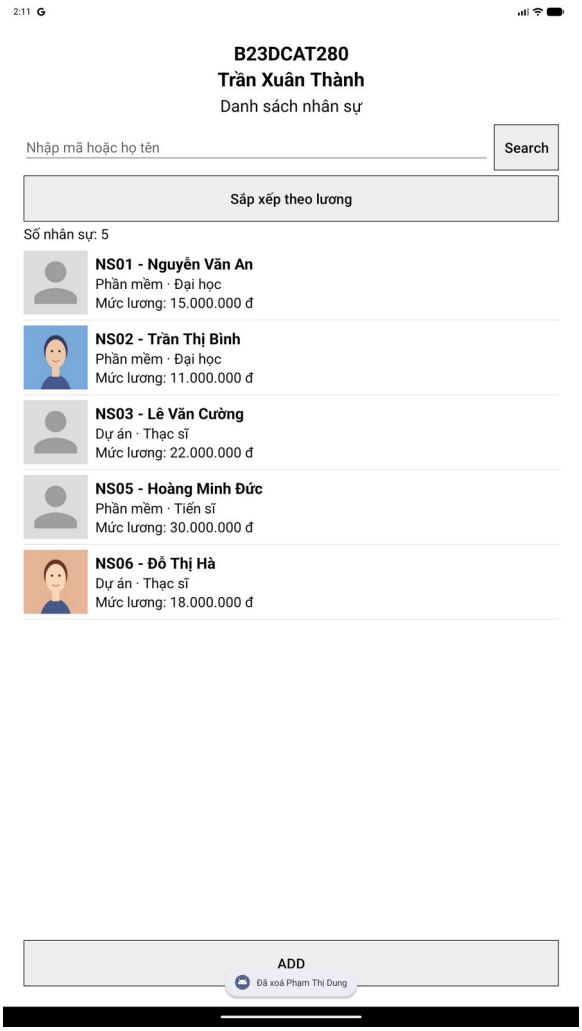


Hình 11. Bấm Update: danh sách cập nhật NS02 (Phần mềm, 11.000.000 đ, có ảnh)

3.4. Xoá nhân sự

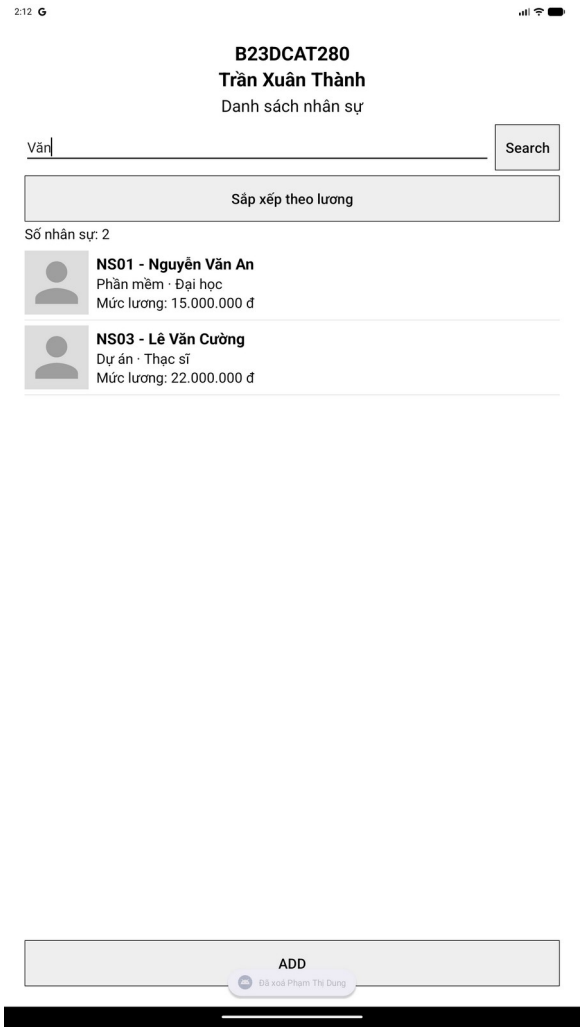


Hình 12. Nhấn giữ dòng NS04: hộp thoại Delete hỏi xác nhận



Hình 13. Bấm Xoá: NS04 biến mất khỏi danh sách, còn 5 nhân sự

3.5. Tìm kiếm

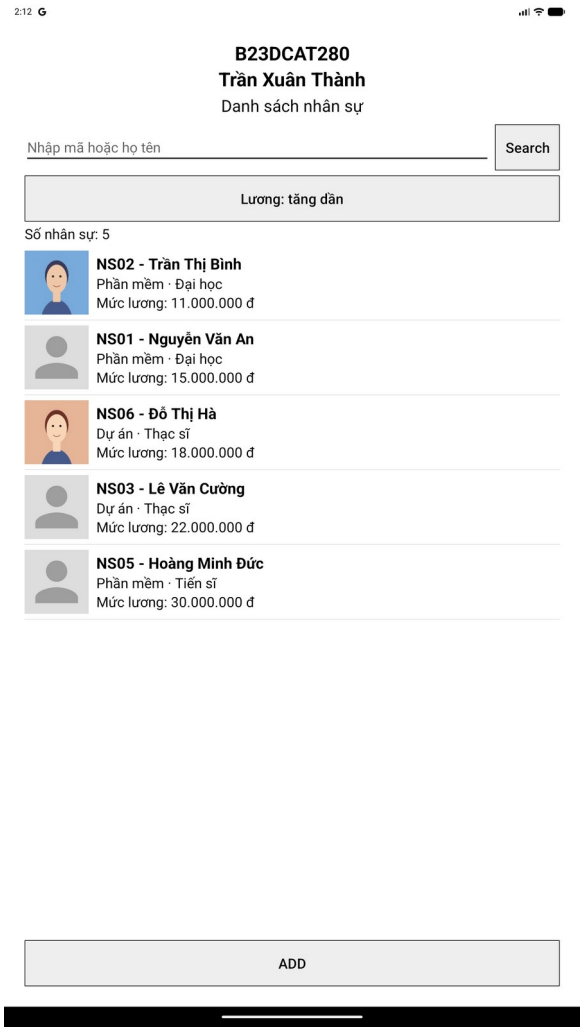


Hình 14. Tìm "Văn": 2 nhân sự có họ tên chứa từ khoá

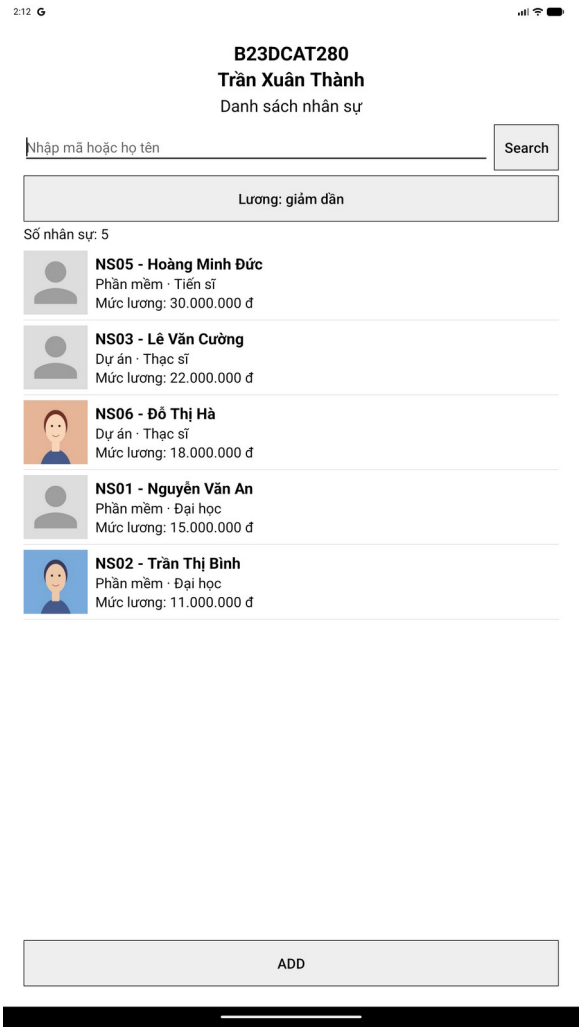


Hình 15. Tìm "xyz": không có nhân sự nào

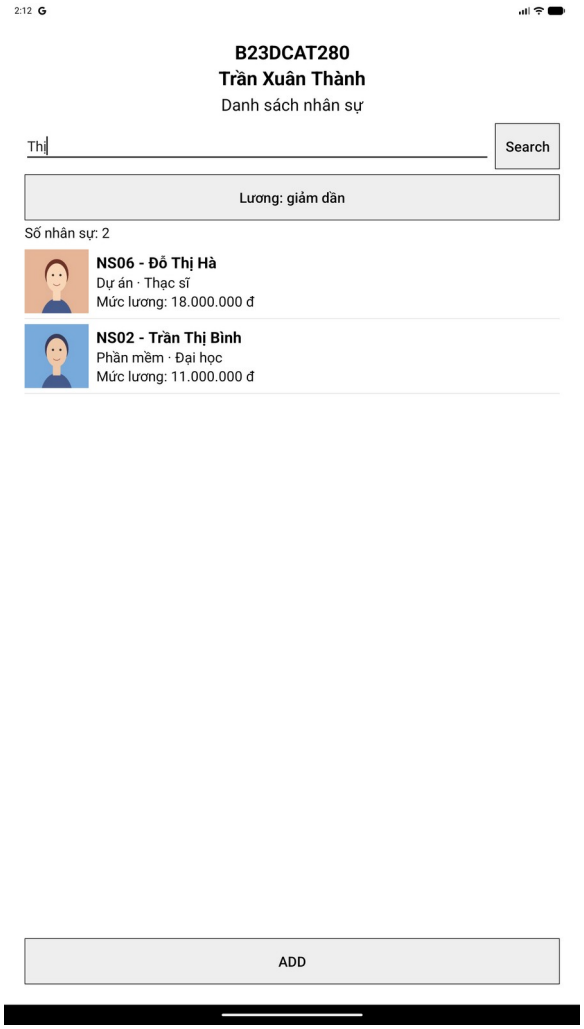
3.6. Sắp xếp theo mức lương



Hình 16. Bấm nút sắp xếp: lương tăng dần



Hình 17. Bấm lần nữa: lương giảm dần



Hình 18. Tìm "Thị" trong khi đang sắp xếp giảm dần: kết quả vẫn giữ thứ tự lương

4. Mã nguồn

DBHelper.java

```
package ptit.cnpm1.tranxuanthanh.quanlynhansu;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

import java.util.ArrayList;
import java.util.List;

// Tạo CSDL quanlynhansu.db, tạo bảng nhansu và các hàm thêm / sửa / xoá / tìm kiếm / sắp xếp
public class DBHelper extends SQLiteOpenHelper {

    private static final String DB_NAME = "quanlynhansu.db";
    private static final int DB_VERSION = 2; // v2: năm sinh -> ngày sinh

    private static final String TABLE = "nhansu";
    private static final String COL_MA = "ma";
    private static final String COL_HO_TEN = "hoten";
```

```
    private static final String COL_NGAY_SINH = "ngaysinh";
    private static final String COL_GIOI_TINH = "gioitinh";
    private static final String COL_QUE_QUAN = "quequan";
    private static final String COL_TRINH_DO = "trinhdo";
    private static final String COL_ANH = "anh";
    private static final String COL_LUONG = "luong";
    private static final String COL_PHONG_BAN = "phongban";


    // Thứ tự sắp xếp danh sách
    public static final int SORT_NONE = 0;
    public static final int SORT_LUONG_TANG = 1;
    public static final int SORT_LUONG_GIAM = 2;


    public DBHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }


    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + TABLE + " ("
                + COL_MA + " TEXT PRIMARY KEY, "
                + COL_HO_TEN + " TEXT NOT NULL, "
                + COL_NGAY_SINH + " TEXT, "
                + COL_GIOI_TINH + " TEXT, "
                + COL_QUE_QUAN + " TEXT, "
                + COL_TRINH_DO + " TEXT, "
                + COL_ANH + " BLOB, "
                + COL_LUONG + " REAL, "
                + COL_PHONG_BAN + " TEXT)");


        // Vài nhân sự mẫu để danh sách có dữ liệu ngay lần chạy đầu
        db.insert(TABLE, null, toValues(new NhanSu("NS01", "Nguyễn Văn An", "1990-05-12",
"Nam", "Hà Nội", "Đại học", null, 15000000, "Phần mềm")));
        db.insert(TABLE, null, toValues(new NhanSu("NS02", "Trần Thị Bình", "1995-08-20",
"Nữ", "Nam Định", "Cao đẳng", null, 9000000, "Hành chính")));
        db.insert(TABLE, null, toValues(new NhanSu("NS03", "Lê Văn Cường", "1988-11-03",
"Nam", "Hải Phòng", "Thạc sĩ", null, 22000000, "Dự án")));
        db.insert(TABLE, null, toValues(new NhanSu("NS04", "Phạm Thị Dung", "1998-02-14",
"Nữ", "Thanh Hóa", "Đại học", null, 12000000, "Kỹ thuật")));
        db.insert(TABLE, null, toValues(new NhanSu("NS05", "Hoàng Minh Đức", "1992-07-30",
"Nam", "Nghệ An", "Tiến sĩ", null, 30000000, "Phần mềm")));
    }


    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE);
        onCreate(db);
    }


    // Danh sách nhân sự có mã hoặc họ tên chứa từ khoá (rỗng = tất cả), sắp xếp theo sort
    public List<NhanSu> getAll(String keyword, int sort) {
        String selection = null;
        String[] args = null;
        if (!keyword.isEmpty()) {
            selection = COL_MA + " LIKE ? OR " + COL_HO_TEN + " LIKE ?";
            args = new String[]{"%" + keyword + "%", "%" + keyword + "%"};
        }


        String orderBy;
        if (sort == SORT_LUONG_TANG) {
            orderBy = COL_LUONG + " ASC, " + COL_MA + " ASC";
        } else if (sort == SORT_LUONG_GIAM) {
            orderBy = COL_LUONG + " DESC, " + COL_MA + " ASC";
        } else {
            orderBy = COL_MA + " ASC";
        }


        List<NhanSu> list = new ArrayList<>();
        SQLiteDatabase db = getReadableDatabase();
        try (Cursor c = db.query(TABLE, null, selection, args, null, null, orderBy)) {
            while (c.moveToNext()) {
```

```
                list.add(fromCursor(c));
            }
        }
        return list;
    }

    // Một nhân sự theo mã, null nếu không có
    public NhanSu getByMa(String ma) {
        SQLiteDatabase db = getReadableDatabase();
        try (Cursor c = db.rawQuery("SELECT * FROM " + TABLE + " WHERE " + COL_MA + " = ?",
new String[]{ma})) {
            return c.moveToFirst() ? fromCursor(c) : null;
        }
    }

    // false nếu thêm không được (vd trùng mã)
    public boolean insert(NhanSu ns) {
        return getWritableDatabase().insert(TABLE, null, toValues(ns)) != -1;
    }

    public boolean update(NhanSu ns) {
        return getWritableDatabase().update(TABLE, toValues(ns), COL_MA + " = ?", new String[]
{ns.getMa()}) > 0;
    }

    public boolean delete(String ma) {
        return getWritableDatabase().delete(TABLE, COL_MA + " = ?", new String[]{ma}) > 0;
    }

    private static ContentValues toValues(NhanSu ns) {
        ContentValues values = new ContentValues();
        values.put(COL_MA, ns.getMa());
        values.put(COL_HO_TEN, ns.getHoTen());
        values.put(COL_NGAY_SINH, ns.getNgaySinh());
        values.put(COL_GIOI_TINH, ns.getGioiTinh());
        values.put(COL_QUE_QUAN, ns.getQueQuan());
        values.put(COL_TRINH_DO, ns.getTrinhDo());
        values.put(COL_ANH, ns.getAnh());
        values.put(COL_LUONG, ns.getLuong());
        values.put(COL_PHONG_BAN, ns.getPhongBan());
        return values;
    }

    private static NhanSu fromCursor(Cursor c) {
        return new NhanSu(
                c.getString(c.getColumnIndexOrThrow(COL_MA)),
                c.getString(c.getColumnIndexOrThrow(COL_HO_TEN)),
                c.getString(c.getColumnIndexOrThrow(COL_NGAY_SINH)),
                c.getString(c.getColumnIndexOrThrow(COL_GIOI_TINH)),
                c.getString(c.getColumnIndexOrThrow(COL_QUE_QUAN)),
                c.getString(c.getColumnIndexOrThrow(COL_TRINH_DO)),
                c.getBlob(c.getColumnIndexOrThrow(COL_ANH)),
                c.getDouble(c.getColumnIndexOrThrow(COL_LUONG)),
                c.getString(c.getColumnIndexOrThrow(COL_PHONG_BAN)));
    }
}
```

NhanSu.java

```
package ptit.cnpm1.tranxuanthanh.quanlynhansu;

import java.util.Locale;

// Một dòng của bảng nhansu
public class NhanSu {

    private final String ma;
    private final String hoTen;
    private final String ngaySinh; // dạng yyyy-MM-dd để so sánh / sắp xếp được
    private final String gioiTinh;
```

```
    private final String queQuan;
    private final String trinhDo;
    private final byte[] anh; // ảnh JPEG đã thu nhỏ, null nếu chưa có ảnh
    private final double luong;
    private final String phongBan;

    public NhanSu(String ma, String hoTen, String ngaySinh, String gioiTinh, String queQuan,
                  String trinhDo, byte[] anh, double luong, String phongBan) {
        this.ma = ma;
        this.hoTen = hoTen;
        this.ngaySinh = ngaySinh;
        this.gioiTinh = gioiTinh;
        this.queQuan = queQuan;
        this.trinhDo = trinhDo;
        this.anh = anh;
        this.luong = luong;
        this.phongBan = phongBan;
    }

    public String getMa() { return ma; }
    public String getHoTen() { return hoTen; }
    public String getNgaySinh() { return ngaySinh; }
    public String getGioiTinh() { return gioiTinh; }
    public String getQueQuan() { return queQuan; }
    public String getTrinhDo() { return trinhDo; }
    public byte[] getAnh() { return anh; }
    public double getLuong() { return luong; }
    public String getPhongBan() { return phongBan; }

    // "1990-05-12" -> "12/05/1990"
    public static String hienThiNgay(String ngay) {
        String[] p = ngay.split("-");
        return p[2] + "/" + p[1] + "/" + p[0];
    }

    // 15000000 -> "15.000.000 đ"
    public String getLuongText() {
        return String.format(Locale.US, "%,.0f", luong).replace(',', '.') + " đ";
    }
}
```

MainActivity.java

```
package ptit.cnpm1.tranxuanthanh.quanlynhansu;

import android.content.Intent;
import android.os.Bundle;
import android.view.inputmethod.EditorInfo;
import android.view.inputmethod.InputMethodManager;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;

import java.util.ArrayList;
import java.util.List;

// Activity_main: danh sách nhân sự, tìm kiếm, sắp xếp theo lương, xoá (nhấn giữ), mở màn thêm
/ sửa
public class MainActivity extends AppCompatActivity {

    private DBHelper db;
    private final List<NhanSu> list = new ArrayList<>();
    private NhanSuAdapter adapter;

    private EditText edtSearch;
```

```
    private Button btnSort;
    private TextView tvCount;

    private String keyword = "";
    private int sort = DBHelper.SORT_NONE;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        db = new DBHelper(this);

        edtSearch = findViewById(R.id.edtSearch);
        btnSort = findViewById(R.id.btnSort);
        tvCount = findViewById(R.id.tvCount);
        Button btnSearch = findViewById(R.id.btnSearch);
        Button btnAdd = findViewById(R.id.btnAdd);
        ListView lvNhanSu = findViewById(R.id.lvNhanSu);

        adapter = new NhanSuAdapter(this, list);
        lvNhanSu.setAdapter(adapter);
        lvNhanSu.setEmptyView(findViewById(R.id.tvEmpty));

        // (V) Tìm kiếm theo mã hoặc họ tên
        btnSearch.setOnClickListener(v -> search());
        edtSearch.setOnEditorActionListener((v, actionId, event) -> {
            if (actionId == EditorInfo.IME_ACTION_SEARCH) {
                search();
                return true;
            }
            return false;
        });

        // Sắp xếp theo lương: bấm lần lượt đổi tăng dần <-> giảm dần
        btnSort.setOnClickListener(v -> {
            sort = sort == DBHelper.SORT_LUONG_TANG ? DBHelper.SORT_LUONG_GIAM :
DBHelper.SORT_LUONG_TANG;
            btnSort.setText(sort == DBHelper.SORT_LUONG_TANG ? R.string.btn_sort_asc :
R.string.btn_sort_desc);
            loadData();
        });

        // (II) ADD mở Activity_add
        btnAdd.setOnClickListener(v -> startActivity(new Intent(this, AddActivity.class)));

        // (III) Bấm 1 dòng mở Activity_detail để sửa
        lvNhanSu.setOnItemClickListener((parent, view, position, id) -> {
            Intent intent = new Intent(this, DetailActivity.class);
            intent.putExtra(DetailActivity.EXTRA_MA, list.get(position).getMa());
            startActivity(intent);
        });

        // (IV) Nhấn giữ 1 dòng để xoá
        lvNhanSu.setOnItemLongClickListener((parent, view, position, id) -> {
            confirmDelete(list.get(position));
            return true;
        });
    }

    // Quay về từ màn thêm / sửa thì đọc lại DB để danh sách luôn mới (refresh)
    @Override
    protected void onResume() {
        super.onResume();
        loadData();
    }

    @Override
```

```
    protected void onDestroy() {
        db.close();
        super.onDestroy();
    }

    private void search() {
        keyword = edtSearch.getText().toString().trim();
        InputMethodManager imm = getSystemService(InputMethodManager.class);
        imm.hideSoftInputFromWindow(edtSearch.getWindowToken(), 0);
        loadData();
    }

    private void confirmDelete(NhanSu ns) {
        new AlertDialog.Builder(this)
                .setTitle(R.string.title_delete)
                .setMessage(getString(R.string.msg_delete, ns.getMa(), ns.getHoTen()))
                .setPositiveButton(R.string.btn_delete, (dialog, which) -> {
                    db.delete(ns.getMa());
                    Toast.makeText(this, getString(R.string.da_xoa, ns.getHoTen()),
Toast.LENGTH_SHORT).show();
                    loadData();
                })
                .setNegativeButton(R.string.btn_cancel, null)
                .show();
    }

    private void loadData() {
        list.clear();
        list.addAll(db.getAll(keyword, sort));
        adapter.notifyDataSetChanged();
        tvCount.setText(getString(R.string.so_nhan_su, list.size()));
    }
}
```

NhanSuAdapter.java

```
package ptit.cnpm1.tranxuanthanh.quanlynhansu;

import android.content.Context;
import android.graphics.BitmapFactory;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ArrayAdapter;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.annotation.NonNull;

import java.util.List;

// Đổ từng NhanSu vào item_nhansu.xml cho ListView ở Activity_main
public class NhanSuAdapter extends ArrayAdapter<NhanSu> {

    public NhanSuAdapter(Context context, List<NhanSu> list) {
        super(context, 0, list);
    }

    @NonNull
    @Override
    public View getView(int position, View convertView, @NonNull ViewGroup parent) {
        if (convertView == null) {
            convertView = LayoutInflater.from(getContext()).inflate(R.layout.item_nhansu,
parent, false);
        }
        NhanSu ns = getItem(position);

        ImageView imgAnh = convertView.findViewById(R.id.imgAnh);
        if (ns.getAnh() != null) {
```

```
            imgAnh.setImageBitmap(BitmapFactory.decodeByteArray(ns.getAnh(), 0,
ns.getAnh().length));
        } else {
            imgAnh.setImageResource(R.drawable.ic_person);
        }

        TextView tvTen = convertView.findViewById(R.id.tvTen);
        TextView tvPhongBan = convertView.findViewById(R.id.tvPhongBan);
        TextView tvLuong = convertView.findViewById(R.id.tvLuong);
        tvTen.setText(ns.getMa() + " - " + ns.getHoTen());
        tvPhongBan.setText(ns.getPhongBan() + " · " + ns.getTrinhDo());
        tvLuong.setText(getContext().getString(R.string.lbl_luong) + ": " +
ns.getLuongText());
        return convertView;
    }
}
```

AddActivity.java

```
package ptit.cnpm1.tranxuanthanh.quanlynhansu;

import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

// Activity_add: nhập nhân sự mới, Submit ghi vào DB rồi quay về danh sách, Reset xoá trắng
form
public class AddActivity extends AppCompatActivity {

    private DBHelper db;
    private NhanSuForm form;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_add);

        db = new DBHelper(this);
        form = new NhanSuForm(this);

        Button btnSubmit = findViewById(R.id.btnSubmit);
        Button btnReset = findViewById(R.id.btnReset);
        btnSubmit.setOnClickListener(v -> submit());
        btnReset.setOnClickListener(v -> form.reset());
    }

    @Override
    protected void onDestroy() {
        db.close();
        super.onDestroy();
    }

    private void submit() {
        NhanSu ns = form.read();
        if (ns == null) return;

        if (db.getByMa(ns.getMa()) != null || !db.insert(ns)) {
            form.edtMa.setError(getString(R.string.err_trung_ma, ns.getMa()));
            form.edtMa.requestFocus();
            return;
        }
        Toast.makeText(this, getString(R.string.da_them, ns.getHoTen()),
Toast.LENGTH_SHORT).show();
        finish();
    }
}
```

DetailActivity.java

```
package ptit.cnpm1.tranxuanthanh.quanlynhansu;

import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

// Activity_detail: xem và sửa 1 nhân sự, Update ghi vào DB rồi quay về danh sách
public class DetailActivity extends AppCompatActivity {

    public static final String EXTRA_MA = "ma";

    private DBHelper db;
    private NhanSuForm form;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_detail);

        db = new DBHelper(this);
        form = new NhanSuForm(this);

        NhanSu ns = db.getByMa(getIntent().getStringExtra(EXTRA_MA));
        if (ns == null) {
            finish();
            return;
        }
        form.fill(ns);
        // Mã là khoá chính nên không cho sửa
        form.edtMa.setEnabled(false);

        Button btnUpdate = findViewById(R.id.btnUpdate);
        btnUpdate.setOnClickListener(v -> update());
    }

    @Override
    protected void onDestroy() {
        db.close();
        super.onDestroy();
    }

    private void update() {
        NhanSu ns = form.read();
        if (ns == null) return;

        db.update(ns);
        Toast.makeText(this, getString(R.string.da_sua, ns.getHoTen()),
Toast.LENGTH_SHORT).show();
        finish();
    }
}
```

NhanSuForm.java

```
package ptit.cnpm1.tranxuanthanh.quanlynhansu;

import android.app.DatePickerDialog;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.net.Uri;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
```

```
import android.widget.RadioButton;
import android.widget.RadioGroup;
import android.widget.Spinner;
import android.widget.Toast;


import androidx.activity.ComponentActivity;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;


import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.util.Arrays;
import java.util.Calendar;
import java.util.Locale;


// Các ô nhập của form_nhansu.xml, dùng chung cho Activity_add và Activity_detail:
// điền dữ liệu có sẵn, xoá trắng, chọn ảnh, đọc và kiểm tra dữ liệu nhập
public class NhanSuForm {


    // Ảnh lưu vào DB được thu nhỏ để cạnh dài nhất không quá 256px
    private static final int ANH_MAX = 256;


    private final ComponentActivity activity;
    final EditText edtMa;
    private final EditText edtHoTen, edtNgaySinh, edtQueQuan, edtLuong;
    private final RadioGroup rgGioiTinh;
    private final RadioButton rbNam, rbNu;
    private final Spinner spTrinhDo, spPhongBan;
    private final ImageView imgAnh;
    private String ngaySinh; // yyyy-MM-dd, null nếu chưa chọn
    private byte[] anh;


    // Phải tạo trong onCreate vì registerForActivityResult chỉ gọi được trước khi activity
chạy
    public NhanSuForm(ComponentActivity activity) {
        this.activity = activity;
        edtMa = activity.findViewById(R.id.edtMa);
        edtHoTen = activity.findViewById(R.id.edtHoTen);
        edtNgaySinh = activity.findViewById(R.id.edtNgaySinh);
        edtQueQuan = activity.findViewById(R.id.edtQueQuan);
        edtLuong = activity.findViewById(R.id.edtLuong);
        rgGioiTinh = activity.findViewById(R.id.rgGioiTinh);
        rbNam = activity.findViewById(R.id.rbNam);
        rbNu = activity.findViewById(R.id.rbNu);
        spTrinhDo = activity.findViewById(R.id.spTrinhDo);
        spPhongBan = activity.findViewById(R.id.spPhongBan);
        imgAnh = activity.findViewById(R.id.imgAnh);
        spTrinhDo.setAdapter(spinnerAdapter(R.array.trinh_do));
        spPhongBan.setAdapter(spinnerAdapter(R.array.phong_ban));


        // Ô ngày sinh không gõ được, bấm vào thì mở lịch chọn ngày
        edtNgaySinh.setOnClickListener(v -> chonNgaySinh());


        // Nút Chọn ảnh mở trình chọn ảnh của máy, chọn xong thì thu nhỏ và hiện lên ImageView
        ActivityResultLauncher<String> chonAnh = activity.registerForActivityResult(
                new ActivityResultContracts.GetContent(), uri -> {
                    if (uri != null) setAnh(uri);
                });
        Button btnChonAnh = activity.findViewById(R.id.btnChonAnh);
        btnChonAnh.setOnClickListener(v -> chonAnh.launch("image/*"));
    }


    public void fill(NhanSu ns) {
        edtMa.setText(ns.getMa());
        edtHoTen.setText(ns.getHoTen());
        setNgaySinh(ns.getNgaySinh());
        rgGioiTinh.check(rbNu.getText().toString().equals(ns.getGioiTinh()) ? R.id.rbNu :
R.id.rbNam);
        edtQueQuan.setText(ns.getQueQuan());
```

```
        selectSpinner(spTrinhDo, R.array.trinh_do, ns.getTrinhDo());
        edtLuong.setText(String.format(Locale.US, "%.0f", ns.getLuong()));
        selectSpinner(spPhongBan, R.array.phong_ban, ns.getPhongBan());
        anh = ns.getAnh();
        showAnh();
    }

    // Xoá trắng mọi ô, kể cả thông báo lỗi còn treo từ lần Submit trước
    public void reset() {
        for (EditText edt : new EditText[]{edtMa, edtHoTen, edtNgaySinh, edtQueQuan,
edtLuong}) {
            edt.setText("");
            edt.setError(null);
        }
        ngaySinh = null;
        rgGioiTinh.check(R.id.rbNam);
        spTrinhDo.setSelection(0);
        spPhongBan.setSelection(0);
        anh = null;
        showAnh();
        edtMa.requestFocus();
    }

    // Đọc form; ô nào sai thì báo lỗi ngay tại ô đó và trả về null
    public NhanSu read() {
        String ma = edtMa.getText().toString().trim();
        String hoTen = edtHoTen.getText().toString().trim();
        String queQuan = edtQueQuan.getText().toString().trim();
        String luongText = edtLuong.getText().toString().trim();

        if (ma.isEmpty()) return loi(edtMa, activity.getString(R.string.err_trong,
activity.getString(R.string.lbl_ma)));
        if (hoTen.isEmpty()) return loi(edtHoTen, activity.getString(R.string.err_trong,
activity.getString(R.string.lbl_hoten)));

        if (ngaySinh == null) return loi(edtNgaySinh, activity.getString(R.string.err_trong,
activity.getString(R.string.lbl_ngaysinh)));

        if (queQuan.isEmpty()) return loi(edtQueQuan, activity.getString(R.string.err_trong,
activity.getString(R.string.lbl_quequan)));

        double luong;
        try {
            luong = Double.parseDouble(luongText);
        } catch (NumberFormatException e) {
            luong = 0;
        }
        if (luong <= 0) return loi(edtLuong, activity.getString(R.string.err_luong));

        String gioiTinh = rgGioiTinh.getCheckedRadioButtonId() == R.id.rbNu
                ? rbNu.getText().toString() : rbNam.getText().toString();
        return new NhanSu(ma.toUpperCase(), hoTen, ngaySinh, gioiTinh, queQuan,
                spTrinhDo.getSelectedItem().toString(), anh, luong,
                spPhongBan.getSelectedItem().toString());
    }

    private NhanSu loi(EditText edt, String message) {
        edt.setError(message);
        edt.requestFocus();
        // Ô ngày sinh không nhận focus nên không hiện bóng lỗi, báo thêm bằng Toast
        if (!edt.isFocusable()) Toast.makeText(activity, message, Toast.LENGTH_SHORT).show();
        return null;
    }

    // Lịch mở sẵn ở ngày đang chọn, chưa chọn thì ở 01/01 của năm cách đây 25 năm; không cho
chọn ngày tương lai
    private void chonNgaySinh() {
        int nam, thang, ngay;
        if (ngaySinh != null) {
            String[] p = ngaySinh.split("-");
```

```
            nam = Integer.parseInt(p[0]);
            thang = Integer.parseInt(p[1]) - 1;
            ngay = Integer.parseInt(p[2]);
        } else {
            nam = Calendar.getInstance().get(Calendar.YEAR) - 25;
            thang = Calendar.JANUARY;
            ngay = 1;
        }
        DatePickerDialog dialog = new DatePickerDialog(activity,
                (view, y, m, d) -> setNgaySinh(String.format(Locale.US, "%04d-%02d-%02d", y, m
+ 1, d)),
                nam, thang, ngay);
        dialog.getDatePicker().setMaxDate(System.currentTimeMillis());
        dialog.show();
    }

    private void setNgaySinh(String ngay) {
        ngaySinh = ngay;
        edtNgaySinh.setText(NhanSu.hienThiNgay(ngay));
        edtNgaySinh.setError(null);
    }

    // Spinner dùng spinner_item.xml để cỡ chữ theo dimens (tablet to hơn)
    private ArrayAdapter<String> spinnerAdapter(int arrayId) {
        ArrayAdapter<String> adapter = new ArrayAdapter<>(activity, R.layout.spinner_item,
                activity.getResources().getStringArray(arrayId));
        adapter.setDropDownViewResource(R.layout.spinner_item);
        return adapter;
    }

    private void selectSpinner(Spinner spinner, int arrayId, String value) {
        int index =
Arrays.asList(activity.getResources().getStringArray(arrayId)).indexOf(value);
        spinner.setSelection(Math.max(index, 0));
    }

    private void showAnh() {
        if (anh != null) {
            imgAnh.setImageBitmap(BitmapFactory.decodeByteArray(anh, 0, anh.length));
        } else {
            imgAnh.setImageResource(R.drawable.ic_person);
        }
    }

    // Đọc ảnh vừa chọn, thu nhỏ và nén JPEG để lưu vào cột BLOB
    private void setAnh(Uri uri) {
        try {
            BitmapFactory.Options options = new BitmapFactory.Options();
            options.inJustDecodeBounds = true;
            try (InputStream in = activity.getContentResolver().openInputStream(uri)) {
                BitmapFactory.decodeStream(in, null, options);
            }
            // Giảm độ phân giải ngay khi đọc để ảnh chụp lớn không làm tràn bộ nhớ
            int sample = 1;
            while (Math.max(options.outWidth, options.outHeight) / (sample * 2) >= ANH_MAX) {
                sample *= 2;
            }
            options = new BitmapFactory.Options();
            options.inSampleSize = sample;
            Bitmap bitmap;
            try (InputStream in = activity.getContentResolver().openInputStream(uri)) {
                bitmap = BitmapFactory.decodeStream(in, null, options);
            }
            if (bitmap == null) throw new IOException("decode failed");

            float scale = Math.min(1f, (float) ANH_MAX / Math.max(bitmap.getWidth(),
bitmap.getHeight()));
            Bitmap small = Bitmap.createScaledBitmap(bitmap,
                    Math.round(bitmap.getWidth() * scale), Math.round(bitmap.getHeight() *
scale), true);
            ByteArrayOutputStream out = new ByteArrayOutputStream();
            small.compress(Bitmap.CompressFormat.JPEG, 85, out);
```

```
            anh = out.toByteArray();
            showAnh();
        } catch (IOException | SecurityException e) {
            Toast.makeText(activity, R.string.err_anh, Toast.LENGTH_SHORT).show();
        }
    }
}
```